Workgroup: Network Management Research Group
Internet-Draft: draft-cui-nmrg-auto-test-02
Published: 29 June 2026
Intended Status: Informational
Expires: 31 December 2026
Authors:
Y. Cui
Tsinghua University
Y. Wei
Tsinghua University
K. Chi
Tsinghua University
X. Xie
Tsinghua University
S. Prabhu
Nokia

A Framework for AI-Assisted Network Protocol Testing from Specifications

Abstract

Network protocol testing is essential for validating that implementations conform to their specifications. Traditional testing approaches rely heavily on manual effort or protocol-specific models that are expensive to build and difficult to reuse as specifications evolve and new protocols emerge.

This document describes a framework for AI-assisted network protocol testing that decomposes the testing workflow into six stages: structured protocol representation, coverage scoping, test case generation, executable artifact generation, test execution, and feedback-based refinement. The framework emphasizes explicit handoffs between stages, keeping the workflow auditable and traceable to the specification text.

The document discusses the design motivations and trade-offs behind the framework, presents illustrative cases drawn from routing protocol testing, and identifies open research challenges.

The RFC Editor will remove this note

About This Document

This note is to be removed before publishing as an RFC.

The latest revision of this draft can be found at https://datatracker.ietf.org/doc/draft-cui-nmrg-auto-test/. Status information for this document may be found at https://datatracker.ietf.org/doc/draft-cui-nmrg-auto-test/.

Discussion of this document takes place on the NMRG Research Group mailing list (mailto:nmrg@ietf.org), which is archived at https://datatracker.ietf.org/rg/nmrg/. Subscribe at https://www.ietf.org/mailman/listinfo/nmrg/.

Source for this draft and an issue tracker can be found at https://github.com/WheaterW/draft-cui-nmrg-auto-test.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 31 December 2026.

Copyright Notice

Table of Contents

1. Introduction

Network protocol testing is used to validate whether an implementation behaves according to the semantics defined by protocol specifications. Such testing is required during device development, interoperability validation, procurement evaluation, regression testing, and operational troubleshooting.

Traditional protocol testing remains largely manual. Engineers read long specifications, identify test points, design topologies, write test procedures, create DUT configurations and tester scripts, execute tests, and inspect results. Model-based approaches can automate parts of this process, but they often depend on protocol-specific models that are expensive to build and difficult to reuse across new or rapidly evolving protocols.

Recent advances in artificial intelligence, especially Large Language Models (LLMs), create new opportunities for automating parts of this workflow. However, protocol testing is not a generic text-to-code task. A protocol test begins with requirements distributed across specification text

and often ends as a coordinated set of tester actions, DUT configurations, timing assumptions, packet observations, and result oracles. Small losses of protocol semantics across these stages can create invalid tests or misleading failure reports.

This document therefore focuses on a framework question: how can a testing workflow carry test-relevant information from protocol specifications to executable test artifacts while remaining auditable and controllable? The framework decomposes the workflow into structured protocol representation, coverage scoping, test case generation, executable artifact generation, test execution, and feedback-based refinement. At each stage boundary, the important handoff is not only the generated artifact, but also the protocol semantics, assumptions, and review points that are passed to the next stage.

2. Problem Scope and Assumptions

This document focuses on specification-derived protocol testing. The primary input is a protocol specification, such as an RFC or a related update document. The target outputs are test cases, tester scripts, DUT configurations, execution results, and reports that can be traced back to the specification.

The framework is mainly intended for conformance, functional, robustness, regression, and related protocol-behavior tests. It does not attempt to cover all implementation-specific defects. Bugs in local management channels, proprietary command-line behavior, vendor-specific configuration subsystems, or non-standard extensions can require additional input documents such as vendor manuals, YANG models, implementation notes, or operational policies.

The framework also assumes that AI-assisted components are not trusted as fully autonomous decision makers. Human review remains important at workflow boundaries, especially for structured representation inspection, coverage scope acceptance, test oracle validation, executable artifact validation, and final bug confirmation. The intended role of automation is to reduce repetitive expert effort while preserving traceability and expert control.

This document does not define a protocol, a test case format, or a maturity-level classification.

3. Definitions and Acronyms

DUT: Device Under Test

Tester: A device with sufficient network protocol functionality and test-control capabilities to execute test cases. It can generate and receive test-specific packets or traffic, emulate target network behaviors, collect observations, and analyze results.

LLM: Large Language Model

AI Agent: A system that can assist with or drive parts of multi-step workflows by using tools and making decisions based on feedback, subject to configured constraints and review points.

API: Application Programming Interface

CLI: Command Line Interface

Test Case: A specification of conditions and inputs to evaluate a protocol behavior.

Tester Script: An executable program or sequence of instructions that controls a protocol tester to generate test traffic, interact with the DUT according to a specified test case, and collect relevant observations for result evaluation.

4. Background

Protocol testing is required during device development, where vendors verify that their implementations conform to protocol specifications, and during procurement evaluation, where third-party organizations perform black-box conformance testing against a neutral standard. Both scenarios share a common set of elements that any testing framework must address.

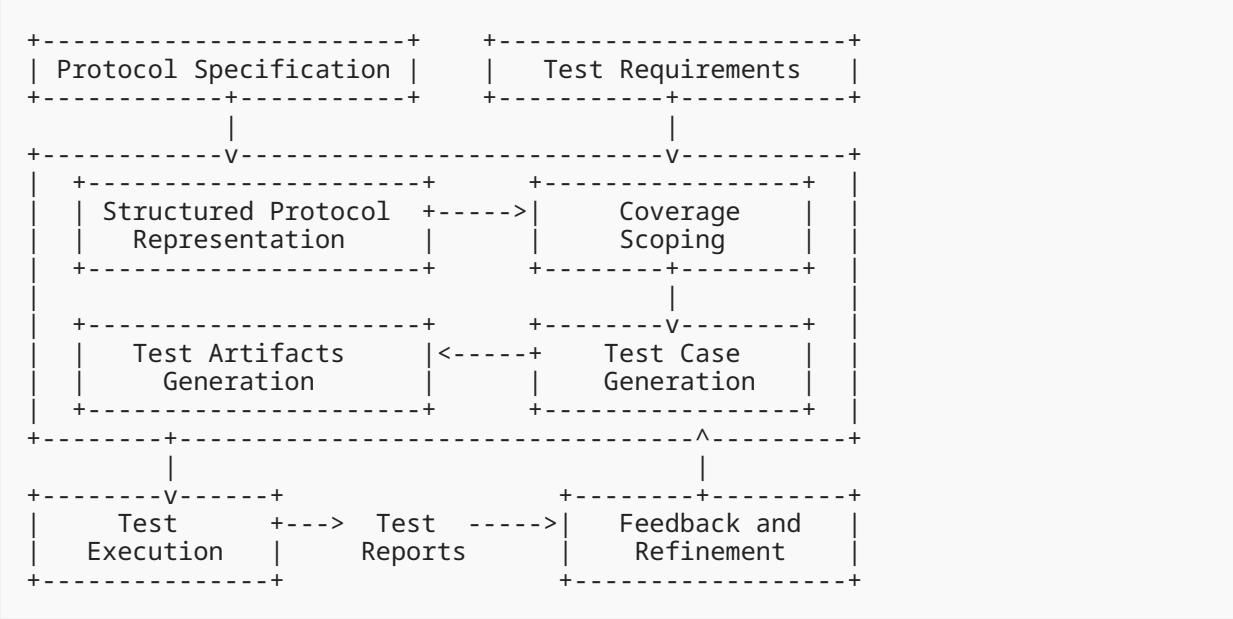
A Device Under Test (DUT) can be a physical network device such as a switch, router, or firewall, or a virtual network device such as an FRRouting (FRR) instance [FRRouting]. A protocol tester is a device with sufficient protocol functionality to generate test traffic, receive and analyze packets, emulate target network behaviors, and produce test results. Testers can typically be controlled via scripts.

Protocol test cases specify the conditions and inputs needed to evaluate a protocol behavior. They cover categories such as conformance, functional, and performance tests, and each test case includes a test topology, step-by-step procedures, expected results, and configuration parameters. Each test case requires a network topology; in batch testing, it is common to construct a minimal common topology that satisfies the requirements of all test cases in the batch.

Before executing a test case, the DUT is initialized with specific configurations (setup). The DUT configuration can change during the test as dictated by the scenario, and is restored upon completion (teardown). The tester is configured and controlled through scripts that coordinate with the DUT configuration to ensure proper interaction during the test.

5. Framework

The AI-assisted network protocol testing framework is illustrated in the figure below. Test requirements are an external input, provided by the testing team for each campaign.



The framework has six stages:

1. Structured protocol representation: transform specification text into a test-relevant representation.
2. Coverage scoping: reconcile the representation with test requirements to produce an explicit, reviewable coverage scope.
3. Test case generation: expand each included scope item into test templates, parameters, and oracles.
4. Test artifacts generation: translate test cases into coordinated tester scripts and DUT configurations.
5. Test execution: run the generated artifacts in a controlled test environment.
6. Feedback and refinement: analyze failures and decide whether to refine artifacts, scope, or bug reports.

The output of each stage becomes a handoff to the next stage. For this reason, each stage can expose not only generated content, but also source references, assumptions, constraints, and review status.

The six stages support two complementary cycles. In the forward cycle, coverage scoping and test case generation produce the test suite from the specification and the test requirements. In the backward cycle, execution feedback is analyzed to localize the likely source of a failure or coverage gap, starting from concrete execution artifacts and moving toward more abstract scope or representation issues when needed. An AI agent can assist in both cycles, but human oversight remains at the acceptance of the coverage scope, the validation of oracles, and the confirmation of protocol defects.

5.1. Structured Protocol Representation

Protocol specifications are usually written for human readers. They include message formats, state machines, timers, variables, algorithms, exception handling, and normative behavior spread across sections and sometimes across multiple update documents. Directly generating executable tests from such text risks losing details that determine whether a test is valid.

The framework therefore introduces a structured protocol representation as an intermediate artifact between specification text and downstream test generation. The representation is not required to be a complete formal model of the protocol. Instead, it preserves the protocol semantics that are relevant for testing.

Such a representation can include:

- message formats, fields, constraints, and valid or invalid values
- local data structures, timers, counters, and protocol variables
- state machines and state transitions
- event-action rules and packet processing behavior
- protocol algorithms, such as route selection or path computation
- error handling and exception behavior
- relationships among modules, such as which messages trigger which transitions or algorithms
- links back to source specification sections

A practical construction workflow can be organized into three steps.

First, semantic enrichment organizes the specification into sections, subsections, cross-references, normative statements, summaries, and update relationships. This step can combine rule-based extraction with AI-assisted summarization.

Second, module induction groups related text into protocol modules, such as message formats, state machines, algorithms, and event-action rules. Each module remains linked to the source text used to construct it.

Third, formalization serializes each module into a structured representation that can be queried, traversed, reviewed, and used by test generation tools.

Protocol updates require special handling. Update documents often add, modify, or deprecate specific protocol behavior rather than restating a complete protocol. An update-aware representation identifies update points, maps them to existing modules, and expresses the update as a differential change to the base representation.

The main design trade-off is between completeness and usefulness. A fully formal protocol model can be expensive to build and difficult to apply across many protocols. A test-relevant representation is less ambitious, but can be more practical if it preserves the semantics needed to generate valid tests, derive oracles, and trace failures back to specifications.

5.2. Coverage Scoping

A structured protocol representation describes what a protocol specification defines. It does not, by itself, state which of those definitions a particular test campaign intends to cover. Coverage scoping is the activity of selecting, from the representation, the set of protocol behaviors that a test suite will exercise, and of documenting which items are excluded and why.

Coverage scoping takes two inputs: the structured protocol representation, and the test requirements for the campaign. Test requirements are external to the framework; they are provided by the testing team and reflect the purpose of the test campaign (e.g., a procurement specification, a regression policy, or a security audit scope). They define the objectives, constraints, and priorities that guide scoping decisions.

The output is a coverage scope: a structured decision record in which each referenced protocol behavior is marked as included or excluded, assigned a priority, and, when excluded, accompanied by a documented reason. Reasons can include specification deprecation by a later RFC, exclusion by the test objective, hardware or time constraints, or other testability limits. The scope references items in the representation without duplicating their definitions; it adds the decisions that the representation cannot express.

The coverage scope serves three purposes. First, it replaces an unverifiable coverage percentage with an inspectable coverage plan: a human reviewer can assess whether the set of included and excluded behaviors is acceptable for the test campaign's objective. Second, it provides the input to test case generation, so that generation focuses on how to test rather than what to test. Third, during iterative refinement, it provides a controlled basis for deciding whether a newly observed behavior or coverage gap should be added to the campaign scope, remain excluded, or trigger a revision of the documented scoping rationale.

An AI agent can assist in producing an initial coverage scope by traversing the representation, applying the test requirements, and proposing inclusion or exclusion with documented reasoning. Final acceptance of the scope requires human review.

5.3. Test Case Generation

Once a coverage scope has been established, test generation expands each included scope item into test cases. Test points can draw on normative statements, message constraints, state transitions, algorithms, error handling requirements, and update deltas from the representation.

The framework separates test templates from test parameters.

A test template captures the reusable structure of a test:

- test objective
- specification reference
- test topology
- preconditions and static configuration

- test procedure
- expected result or oracle
- variable placeholders

Parameters instantiate those placeholders with concrete values. Parameter generation can include valid values, invalid values, boundary values, timing values, topology variants, and values computed by helper logic. Oracle values can also require computation, especially when expected behavior depends on route selection, timers, path attributes, or other protocol algorithms.

This separation is a key design choice. Templates provide breadth across protocol functions and test scenarios. Parameters provide depth across value spaces and boundary conditions. Equivalence partitioning or similar reduction techniques can then group parameter combinations that exercise the same protocol behavior, reducing redundant execution without discarding meaningful coverage.

Generated test cases still require review. Correctness means that a test reflects the intended protocol semantics. Coverage means that the test suite exercises relevant protocol functions, behaviors, and parameter spaces. Because protocol test cases mix natural language, topology assumptions, configuration fragments, packet observations, and oracles, systematic evaluation of correctness and coverage remains an open research challenge.

5.4. Executable Artifact Generation

Executable artifact generation translates abstract test cases into runnable tester scripts and DUT configurations. This step is difficult because tester actions and DUT configurations are coordinated together. They agree on topology, addresses, protocol parameters, timing, expected packets, and oracle logic.

Directly exposing heterogeneous tester APIs and device-specific configuration mechanisms creates a large and error-prone action space. A useful framework can therefore introduce explicit testbed abstractions, such as:

- a DUT control abstraction for applying configuration, changing runtime state, and collecting diagnostic outputs
- a tester-side logic abstraction for protocol emulation, traffic control, packet capture, and result queries
- an execution abstraction for running tests, collecting logs, and reporting pass/fail outcomes

These abstractions constrain the generation problem and make artifacts easier to validate. Validation covers both syntax and semantics. Syntax validation detects invalid API calls or CLI commands. Semantic validation checks whether the artifacts implement the intended test logic and whether tester and DUT configurations are mutually consistent.

5.5. Test Execution and Feedback

Test execution runs the generated artifacts in a controlled test environment and collects the observations needed for result evaluation.

Execution feedback requires careful interpretation. A failed test does not always indicate a protocol implementation defect. It can be caused by an invalid test case, an incorrect oracle, a generated script error, a DUT configuration issue, a topology problem, or environmental instability. Feedback analysis distinguishes at least four categories:

- likely DUT protocol violation
- invalid or ambiguous test case
- executable artifact error
- test environment or setup failure

A feedback record can be organized into observed inputs, analysis, and refinement actions. Observed inputs can include pass/fail results, execution logs, diagnostic outputs, telemetry collected during the run, and DUT state or configuration snapshots. The analysis maps discrepancies between expected and observed behavior to likely causes, while refinement actions can correct the test case, adjust parameters or timing, update the oracle, or fix tester scripts and DUT configurations.

Human review is most valuable at the boundaries where these categories are decided. In practical deployments, the goal is not to remove experts from the workflow, but to shift their effort from manual artifact construction to targeted review, correction, and final decision-making.

6. Design Considerations

6.1. Structured Workflow Boundaries

End-to-end prompting can produce useful drafts, but it is a weak workflow boundary for protocol testing. It often hides which specification text was used, which constraints were preserved, and which assumptions were introduced. Structured workflow boundaries, with explicit handoffs that carry source references and assumptions, provide a more inspectable and auditable path from specification to executable tests.

6.2. Test-Relevant Protocol Representation

Complete formal models can provide strong guarantees, but they are expensive to build and difficult to maintain across many protocols and update documents. A test-relevant representation makes a weaker claim: it aims to preserve the semantics needed for testing. This is less complete, but more feasible for broad protocol coverage.

6.3. Separation of Templates and Parameters

Generating many concrete tests directly can produce large and redundant suites. Separating templates from parameters allows broad functional coverage and deep parameter exploration to scale independently. The cost is that the framework also manages parameter reduction, oracle computation, and test-suite scheduling.

6.4. Human-in-the-Loop Review

AI-assisted workflows can help analyze execution feedback, but the interpretation of test results should not be fully automated. A failed test may result from a DUT protocol violation, an incorrect oracle, an invalid test case, a generated script error, a configuration issue, or instability in the test environment. The framework therefore keeps experts in the loop for high-impact interpretation and approval decisions.

6.5. Specification-Derived Testing

Specification-derived testing targets behavior defined by protocol specifications. It can miss implementation-specific bugs in management functions, local control channels, vendor extensions, or configuration subsystems. Expanding the input corpus can broaden coverage, but also introduces new ambiguity and source-trust questions.

7. Use Cases

This section presents illustrative use cases that motivate the framework. The cases are not intended to define required behavior for all implementations.

7.1. Update-Aware Testing

Protocol behavior changes over time as RFCs are updated. For example, RFC 9774 [RFC9774] formally deprecated the AS_SET path attribute in BGP. A framework that treats RFC 9774 in isolation can miss the affected base-protocol behavior that remains defined in RFC 4271 [RFC4271].

An update-aware representation can map the RFC 9774 [RFC9774] deprecation to the corresponding BGP path attribute and state-machine rule in the base representation. Test generation can then focus on the changed behavior. The generated test advertises a route containing AS_SET and checks whether the DUT correctly removes it from the AS_PATH. When this test was applied to deployed implementations, several devices were found to still propagate the deprecated attribute, including a widely used open-source routing stack that subsequently fixed the behavior in a later release.

The lesson is that update documents are best represented as differential changes over existing protocol modules, not as standalone specifications.

7.2. Specification-Derived Bug Exposure

Specifications can define behavior that is easy to overlook in manual test suites. RFC 2453 [RFC2453] defines conditions under which a RIP router originates and advertises a default route to neighbors (Section 3.7). A manually curated test suite might verify basic route exchange without exercising this specific condition. A structured representation makes the condition explicit, so that test generation can configure the relevant scenario, capture routing updates, and check whether the expected default route appears. This approach has exposed implementation defects that were not covered by existing manually maintained test suites.

The lesson is that systematic extraction of test points from structured protocol behavior can expose gaps in manually curated test suites, especially for less frequently tested edge cases.

7.3. Parameterized Boundary Testing

Some bugs appear only under specific parameter relationships rather than under a single fixed input. For example, in OSPF, route selection can depend on relative path costs, and DR election depends on relative router priorities. Template-parameter separation supports this class of tests. A template captures the protocol scenario, while parameter instantiation explores relevant value relationships such as equal costs, boundary priority values, and timing orderings. Equivalence partitioning can reduce redundant combinations while preserving representative boundary cases. Across a set of manually classified protocol implementation bugs, parameterized tests detected defects that single-value test instances did not exercise.

The lesson is that test depth often depends on parameter relationships and oracle computation, not only on the number of test cases.

8. Research Challenges

Several research challenges remain open for AI-assisted protocol testing. Each is discussed briefly below.

Representation fidelity: The community lacks widely accepted metrics for whether a structured protocol representation preserves the semantics needed for testing. Without such metrics, comparing alternative representation approaches or determining when a representation is adequate for a given testing objective remains difficult.

Coverage scope validation and completeness: A coverage scope reconciles test requirements with a structured protocol representation, but both are grounded in natural-language specifications for which no machine-readable definition of complete coverage exists. Methods for assessing whether a coverage scope adequately captures the protocol behaviors relevant to a given test objective, and for measuring how completely a derived test suite exercises its stated scope, remain open.

Update-aware testing: RFC updates, extensions, and deprecations require methods to localize specification changes and propagate them through the representation to the affected tests. Current practice largely treats each RFC version as a standalone document, missing opportunities to reuse existing test assets and to focus testing effort on changed behavior.

Cross-vendor artifact abstraction: Tester APIs and device configuration mechanisms differ across vendors and testbeds. Common abstractions for DUT control, tester logic, and execution management are needed to make generated artifacts portable across test environments and to allow human reviewers to assess them without vendor-specific expertise.

Oracle generation and validation: Some expected results can be computed deterministically from protocol algorithms, while others depend on timing, ordering, or implementation-specific behavior. Generating correct oracles for the latter category, and validating oracle correctness when no higher-level oracle exists, remains difficult.

Failure triage: A failed test can indicate a DUT protocol violation, an incorrect test case, a script or configuration error, or a transient environment issue. Reliably distinguishing among these causes, especially when an AI agent participates in artifact generation and its own reasoning may contribute to the failure, remains an open challenge.

Feedback-loop reproducibility: Refinement decisions need enough recorded context to be replayed, audited, and compared across tool versions or test environments. Without reproducible refinement, it is difficult to assess whether a change to the framework or the underlying AI model improves or degrades testing outcomes over time.

Test-suite management: Large generated suites can contain thousands of test cases, requiring methods for reduction, scheduling, topology clustering, prioritization, and regression selection. Managing such suites across protocol updates and test campaigns, without losing coverage of critical behaviors, is an open operational challenge.

Auditability and traceability: When an AI-assisted workflow produces a test result, stakeholders need to trace that result back through executable artifacts, test cases, the coverage scope, the protocol representation, and ultimately to the specification text. End-to-end traceability across all these stages, in a form that is both machine-processable and human-reviewable, remains an open challenge.

Safety and security: Automatically generated code and configuration can disrupt test environments or create misleading reports. Determining the appropriate level of automation versus human control for operations at different risk levels, beyond the general guidance of sandboxing and validation, remains an open question.

9. Conclusion

This document has described a framework for AI-assisted network protocol testing from specifications. The framework provides a structured workflow, from protocol representation through coverage scoping, test generation, execution, and feedback, in which stage boundaries

are explicit, handoffs carry traceability to the specification, and human oversight is applied at the points where errors carry the highest cost. The authors hope this framing helps the NMRG community discuss and advance the research challenges identified in this document.

10. Security Considerations

1. Execution of Unverified Generated Code: Automatically generated test scripts or configurations (e.g., CLI commands, tester control scripts) can include incorrect or harmful instructions that misconfigure devices or disrupt test environments. Mitigation: Validate all generated artifacts, including syntax checking, semantic verification against protocol constraints, and dry-run execution in sandboxed environments.
2. AI-Assisted Component Risks: LLMs and AI agents can produce incorrect or insecure outputs due to their probabilistic nature or prompt manipulation, and the risk compounds when an agent chains multiple tool calls across workflow stages without intermediate review. Mitigation: Apply input sanitization, prompt hardening, and human-in-the-loop validation for critical operations.

11. IANA Considerations

This document has no IANA actions.

12. Informative References

[FRRouting] "FRRouting", n.d., <https://frrouting.org/>.

[RFC2453] Malkin, G., "RIP Version 2", STD 56, RFC 2453, DOI 10.17487/RFC2453, November 1998, <https://www.rfc-editor.org/rfc/rfc2453>.

[RFC4271] Rekhter, Y., Ed., Li, T., Ed., and S. Hares, Ed., "A Border Gateway Protocol 4 (BGP-4)", RFC 4271, DOI 10.17487/RFC4271, January 2006, <https://www.rfc-editor.org/rfc/rfc4271>.

[RFC9774] Kumari, W., Sriram, K., Hannachi, L., and J. Haas, "Deprecation of AS_SET and AS_CONFED_SET in BGP", RFC 9774, DOI 10.17487/RFC9774, May 2025, <https://www.rfc-editor.org/rfc/rfc9774>.

Acknowledgments

This work is supported by the National Key R&D Program of China.

Contributors

Zhen Li
Beijing Xinertel Technology Co., Ltd.
Email: lizhen_fz@xinertel.com

Zhanyou Li
Beijing Xinertel Technology Co., Ltd.
Email: lizy@xinertel.com

Authors' Addresses

Yong Cui
Tsinghua University
Email: cuiyong@tsinghua.edu.cn

Yunze Wei
Tsinghua University
Email: wyz23@mails.tsinghua.edu.cn

Kaiwen Chi
Tsinghua University
Email: ckw24@mails.tsinghua.edu.cn

Xiaohui Xie
Tsinghua University
Email: xiexiaohui@tsinghua.edu.cn

Shailesh Prabhu
Nokia
Email: shailesh.prabhu@nokia.com